

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process States at different stages of execution

Code:

```
geya@LenovoL480: ~/OS/Pra  ×  +  v
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Before fork: Process PID = %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State: Running\n");

        sleep(5);

        printf("Child State: Terminated\n");
    }
    else {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State: Running\n");

        wait(NULL);

        printf("Parent State: Running after child termination\n");
    }

    return 0;
}
```

Output:

```
geya@LenovoL480:~/OS/Practical/Week 3$ ./task1
Before fork: Process PID = 2810
Parent Process
PID = 2810
PPID = 325
State: Running
Child Process
PID = 2811
PPID = 2810
State: Running
Child State: Terminated
Parent State: Running after child termination
geya@LenovoL480:~/OS/Practical/Week 3$ |
```